

Chuleta de ejecuciones

Propósito: Resumen práctico para lanzar y resolver incidencias en las ejecuciones: flujo entre API, Celery, workers/robots, robots específicos y base de datos; soluciones rápidas; checklist y comandos útiles.

Flujo de Ejecuciones

- **Resumen (alto nivel):** API -> Cola (Redis/Rabbit) -> Celery Beat/Producer -> Celery Workers -> Dispatcher/Enqueueur -> Robot genérico -> Robot específico -> Base de datos (estado/resultados)
- **Diagrama compacto:** API -> Celery (task) -> Worker -> Robot -> DB
- **Tareas predefinidas y módulos implicados:**
 - **Altas:** [api.routes.altas] recibe petición -> **enqueueur** crea task -> Celery almacena en broker -> **worker** ejecuta -> **robot/altas/*** realiza pasos -> **database/altas** guarda resultado.
 - **Consultas (enotum):** ruta API -> encola task consulta -> worker levanta **robot/consultas/*** -> consulta externa -> parseo -> DB.
 - **Descargas:** API o scheduler -> encola descarga -> worker ejecuta robot de descargas -> guarda **descarga_result** y ficheros.
 - **Matriculas y puntos:** flujo similar; robot específico **robot/matriculasypuntos** con pasos adicionales de validación.

Puntos clave del flujo

- **API:** validar payload, devolver task id inmediato, registrar request en logs.
- **Broker (Redis):** configurar TTLs y persistencia para evitar pérdidas de tasks; monitorizar memory y latencia.
- **Celery:** separar colas por tipo de tarea (altas, consultas, descargas). Usar `routing_key` para priorizar.
- **Workers:** etiquetar nodos por capacidades (ej: **has_certificate**, **chromedriver**) y usar **-Q** para asignar colas.
- **Robots:** empaquetar dependencias; cada robot debe devolver códigos de estado estandarizados (OK, RETRY, FAIL, FATAL) y mensajes estructurados.
- **DB:** escribir transacciones al final del task; mantener tabla de ejecuciones con estados: PENDING, RUNNING, SUCCESS, RETRY, FAILED.

Soluciones rápidas y 'chapuzas' (workarounds) para mejorar estados y nodos

- **'Chapuzas' para estado zombie:** si un node aparece RUNNING pero no responde, acceder al nodo mediante 'RustDesk' y eliminar ventana de cmd con el worker iniciado.
- **Reparar nodos con drivers de Chrome bloqueados:** limpiar procesos **chromedriver/chrome** en el host y reiniciar el servicio del worker.
- **Eliminar 'stalled' tasks:** Lanzar el script de python playground para eliminar tareas con estado 'RUNNING' y 'PENDING', de manera indiscriminada.

Lanzamiento de robots y requisitos para ajustar nodos

- **Pre-requisitos por robot:**
 - **Broker sano:** conexiones y memoria OK.
 - **Workers suficientes:** número de workers y concurrency calculado.

- **Certificados:** redtrust iniciado y el Windows Tool Tray.
- **Ejemplo de arranque (De manera local):**
 - Iniciar entorno virtual y worker:
 - `source .venv/Scripts/activate` (o `.venv\Scripts\activate` en PowerShell)
 - `celery -A api.worker worker--concurrency=1 --hostname=%NODE_IP%@%COMPUTERNAME% --pool=solo -Q robot_tasks -E --loglevel=info -n %NODE_IP%@%COMPUTERNAME%`
 - `scripts\start.bat`

Monitoreo, Logs y métricas

- **Logs:** centralizar logs en `logs/` por tipo:
`robot/altas/,robot/descargas/,robot/consultas/,robot/matriculasypuntos/,robot/sedejudicial/, /api`. Incluir `task_id` en todas las líneas. ti
- **Métricas clave:** tasks/segundo, retries, tiempo medio por task, errores 500 en APIs, uso de memoria de Redis.

Comandos útiles y snippets

- **Ver tasks pendientes en Redis:** `redis-cli --scan` / usar `celery -A api.worker inspect active`.
- **Revocar task:** `celery -A api.worker control revoke <task_id> --terminate --signal=SIGKILL`.
- **Reenviar task manualmente:** desde shell Python:
 - `from api.enqueue import enqueue_task`
 - `enqueue_task('tipo', payload)`
- **Reiniciar worker (Windows):** detener proceso y relanzar el comando celery.

Casos frecuentes y soluciones rápidas

- **Task tarda demasiado y bloquea recursos:** marcar RUNNING -> RETRY automático con backoff; si afecta a otros, poner en mantenimiento la cola.
- **Errores 500 en llamadas externas:** comprobar estado del nodo master ip: *.8, docker corriendo y revisar logs (archivo o inline de docker)
- **Ficheros no guardados:** revisar permisos y ruta de destino; si falla, mover a carpeta temporal y reintentar escritura.

Notas operativas y recomendaciones

- Mantener colas separadas por tipo y propósito (latency vs batch).
- Robots deben ser idempotentes cuando sea posible.
- Registrar siempre `task_id` y `execution_id` para trazabilidad.
- Automatizar limpiezas periódicas de `logs/` y `files/` para evitar llenado.

Recursos / archivos relevantes

- API y workers: `api/` (ver `api/worker.py`, `api/enqueuer.py`, `api/dispatcher.py`).
 - Robots: `app/robot/` y subcarpetas por tipo.
 - Config: `app/helper/config.py`, `api/config/api_config.py`.
-

Acceso y comandos útiles para Redis y consola

Acceso a consola Redis en Docker:

```
docker exec -it redis redis-cli -h 192.168.184.162 -p 6379
```

Listar claves:

```
KEYS *
```

Listar claves ordenadas (desde shell):

```
docker exec -it redis redis-cli -h 192.168.184.162 -p 6379 KEYS "*" | sort
```

Ver el contenido de una clave según tipo:

- **String:**
`GET <clave>`
- **Hash:**
`HGETALL <clave>`
- **List:**
`LRANGE <clave> 0 -1`
- **Set:**
`SMEMBERS <clave>`

Ver tipo de clave:

```
TYPE <clave>
```

Eliminar una clave:

```
DEL <clave>
```

Eliminar todas las claves que coincidan con un patrón:

```
EVAL "return redis.call('del', unpack(redis.call('keys', 'queue_log:*')))" 0
```

Eliminar todas las claves de Redis:

```
EVAL "return redis.call('del', unpack(redis.call('keys', '*')))" 0
```

Eliminar tasks en estado RUNNING para un nodo específico:

```
EVAL "local h = 'task_registry:<NODO>' local keys = redis.call('HKEYS', h) for i, k in ipairs(keys) do local v = redis.call('HGET', h, k) if v and v:find('\\"status\\": \\"running\\")' then redis.call('HDEL', h, k) end end return true" 0
```

Reemplaza *<NODO>* por el identificador del nodo, por ejemplo: *192.168.184.133@WS-090*

Inspeccionar contenido de todos los task_registry:

```
EVAL "local keys=redis.call('keys','task_registry:*'); local out={}; for _,k in ipairs(keys) do table.insert(out,'== '..k..' =='); local h=redis.call('hgetall',k); for i=1,#h,2 do table.insert(out,' '..h[i]..'': '..h[i+1]); end end; return out" 0
```

Ver procesos celery/python en Windows (PowerShell):

```
Get-Process | Where-Object { $_.ProcessName -match "celery|python" }
```

Extra

Acceder a <http://192.168.184.8:8008/docs>, para futura documentación de la API y usabilidad de la misma.